SOFTWARE ARCHITECTURE · FINAL PROJECT DOCUMENTATION

GYM IT System

Modular Monolith and Microservices — two architectures for the same gym tournament management platform.

Course	Software Architectures
Domain	Gym tournament management — auth, brackets, scoring, leaderboard, notifications
Monolith repository	github.com/Ilagrim2/Gymonolith
Microservices repository	github.com/personnna/software-architecture
Date	June 2026

Team and responsibilities

Team member	Core domain	Architectural responsibility
Yeldana Kadenova	Challenges & leaderboards	Simplicity & UI architecture
Daniil Glazunov	User management — auth, profiles, roles	Security — JWT, AES-256, RBAC
Shattyk Kuziyeva	Tournament engine — brackets & scoring	Scalability & DevOps
Mirgalil Azhmukhambetov	Reporting & notifications	Fault tolerance & data

Contents

1 Introduction

This document describes the architecture of the **GYM IT System**, a web application for managing gym tournaments. The system lets members register and sign in, lets trainers and administrators create tournaments and generate brackets, records and scores matches, keeps a points-based leaderboard, and produces notifications for tournament events.

Following the same approach as the course's reference projects, the system was built in two architectural styles in order to study and compare them. It was first developed as a **modular monolith** for simplicity, development speed and clear domain boundaries, and then re-developed as a set of **microservices** to obtain properties such as independent scalability, fault isolation and independent deployment. The two versions live in two separate repositories:

- **Modular monolith** — `github.com/Ilagrim2/Gymonolith`: a single Flask application whose domains are internal modules sharing one database.
- **Microservices** — `github.com/personnna/software-architecture`: independent Flask services behind an API Gateway, integrated through a RabbitMQ event broker, each owning its data.

Both versions implement the same domain and the same core features, so the comparison isolates the effect of the architecture itself. The remainder of this document covers the requirements, the technology stack, each architecture in detail with diagrams, a direct comparison, the shared security and design patterns, testing, and the team's responsibilities.

2 Requirements and how to run

2.1 Functional requirements

- Members self-register and authenticate; the system issues a signed JWT.
- Three roles — **administrator**, **trainer**, **member** — with different permissions.
- Trainers and administrators create tournaments, register participants and generate brackets.
- Single-elimination brackets are generated; ties are rejected and winners advance automatically.
- Match results update player statistics (played, wins, losses, points, titles) and the leaderboard.
- Members view tournaments, brackets and the leaderboard, and edit their own profile.
- Administrators manage accounts — assign roles and activate or deactivate users.
- Tournament events drive notifications (in-app in the monolith; event-driven in the microservices version).

2.2 How to run

Both projects rely on **Docker** for a consistent, standardised execution. With Docker running, from inside each cloned repository (where the compose file lives):

```
docker compose up --build
```

In the microservices version, Docker Compose builds and runs multiple containers — one per service with its own Dockerfile — plus the databases and the message broker. Entry points:

Version	Open at	Notes
Modular monolith	http://localhost:8000	Single app container + one PostgreSQL container.
Microservices	http://localhost:8000 (API Gateway)	RabbitMQ management UI at http://localhost:15672 (user/pass: gym/gym).

In both versions the **first registered user becomes** `admin`; later users join as `member` until an admin changes their role. Detailed steps are in each repository's `README.md`.

3 Technologies

Both versions deliberately share the same core stack so the architecture is the only variable.

Concern	Technology
Language / framework	Python 3.11 with Flask (Flask-SQLAlchemy ORM).
Database	PostgreSQL 16. One shared database in the monolith; one database per service in the microservices version.
Authentication	Stateless JWT (PyJWT, HS256). Passwords hashed with Werkzeug.
Encryption	AES-256-GCM (cryptography library) for sensitive fields such as phone numbers.
Frontend	Monolith: server-backed Bootstrap 5 + vanilla JS pages. Microservices: static HTML/CSS/JS served by the gateway.
Messaging	RabbitMQ topic exchange (microservices version only) for asynchronous domain events.
Schema management	Flask-Migrate / Alembic migrations (monolith).
Runtime / serving	Gunicorn WSGI server in Docker containers.
Orchestration	Docker Compose for both; Kubernetes manifests for the microservices version.
Testing	Pytest (both); GitHub Actions CI in the microservices version.

4 Architectures

4.1 Modular monolith (Gymonolith)

The system was first built as a **modular monolith**: a single deployable Flask application that is internally divided into well-separated modules, one per domain. The monolith was chosen for its simplicity, maintainability, development speed and clear domain boundaries. All modules run inside one Flask process, share one SQLAlchemy session, and persist to one PostgreSQL database. There is no API Gateway, no message broker and no service-per-database; modules communicate through ordinary Python function calls and shared database transactions.

Modules

The application is assembled by an application factory (`create_app`) that registers one Flask blueprint per module:

Module	Responsibility
auth	JWT generation, login, registration, centralized RBAC decorators.
users	Profile management and administrative user management.

Module	Responsibility
tournaments	Tournament setup, participants, bracket generation and match scoring.
leaderboard	Points, wins, losses, played tournaments and champion titles.
notifications	In-app notifications stored in the same database.
reports	Administrative dashboard aggregates.
web	Bootstrap pages backed by the internal JSON API.

Persistence

A single database holds one schema with the core tables `users`, `tournaments`, `tournament_participants`, `matches`, `leaderboard_entries` and `notifications`. Production schema changes are managed with Flask-Migrate / Alembic. Because everything shares one transaction, notification and leaderboard updates happen **synchronously** within the same database transaction as the tournament action that triggered them.

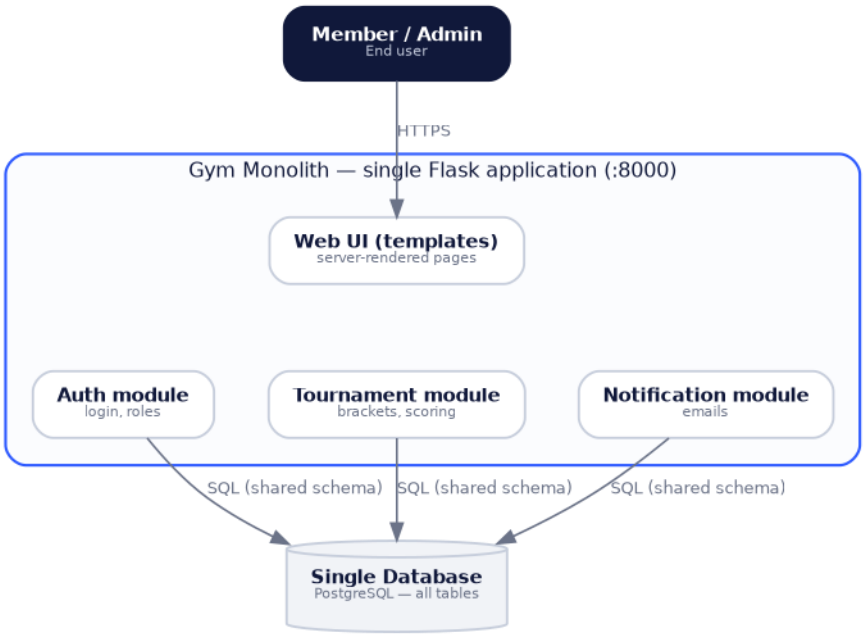

Figure 1 — Modular monolith: one Flask application with internal modules over a single database.

4.2 Microservices (personnna/software-architecture)

The system was then re-developed as **microservices** to enforce strict domain boundaries at the deployment level and to allow each domain to scale, deploy and fail independently. Responsibilities are split into small Flask services, each owning a single domain and its own database, integrated through a single API Gateway and a RabbitMQ broker. Figure 2 shows the overall architecture in layers — deployment, API layer, message broker, microservices and databases.

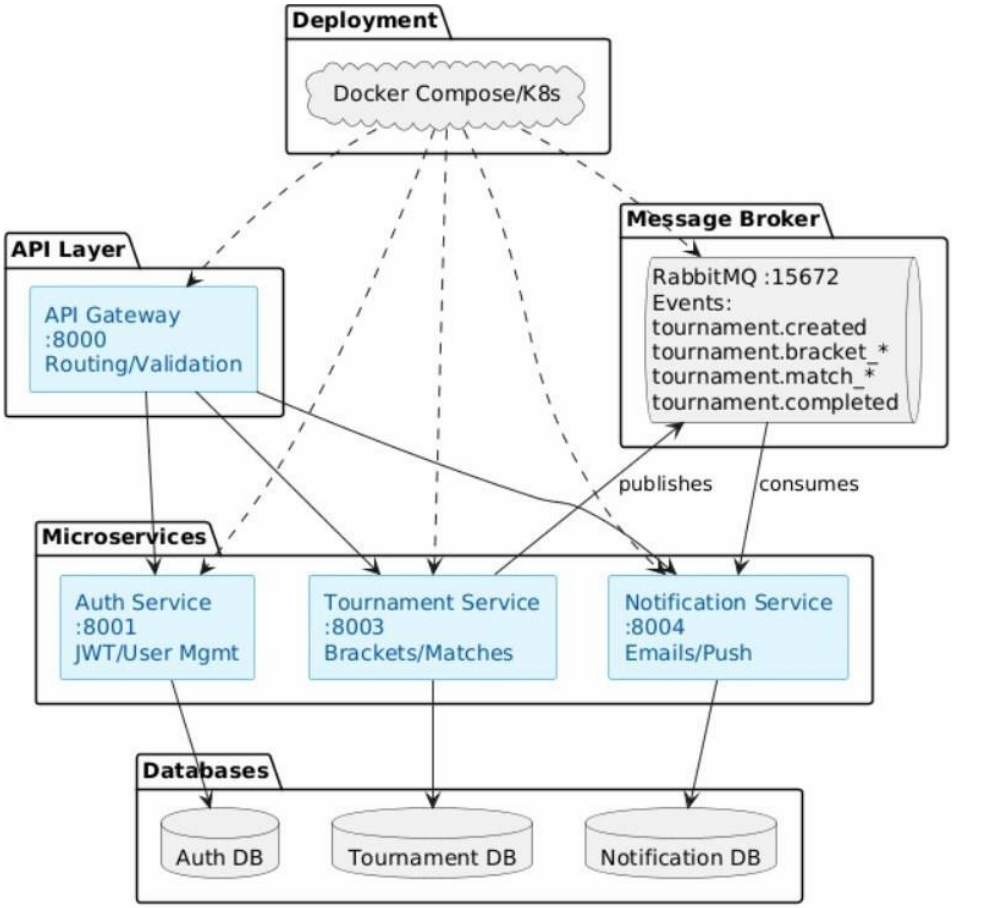

Figure 2 — Microservices architecture overview (deployment, gateway, broker, services and databases).

System context

Two human actors use the system: **members** join tournaments and follow brackets and results, while **administrators and trainers** create tournaments, manage participants and record scores. Tournament events are projected to notifications.

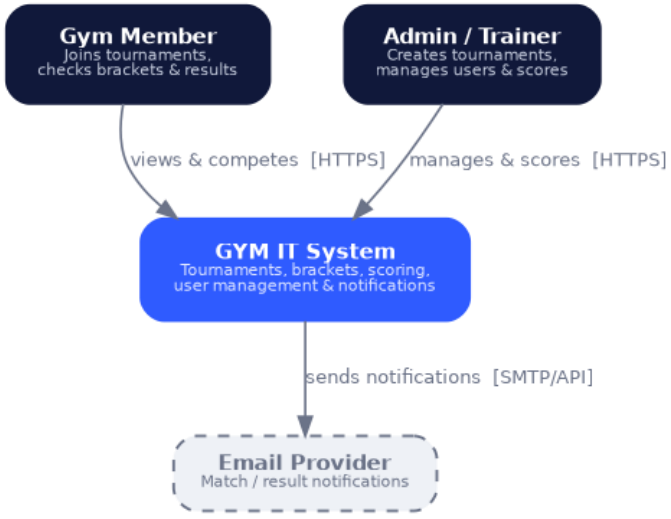

Figure 3 — C4 level 1: system context.

Containers and services

Clients talk only to the API Gateway, which routes `/api/*` calls to the Auth and Tournament services and serves the web frontend. The Tournament service publishes events to RabbitMQ, which delivers them to the

Notification service, and calls the Auth service with a service token to update player statistics after a result.

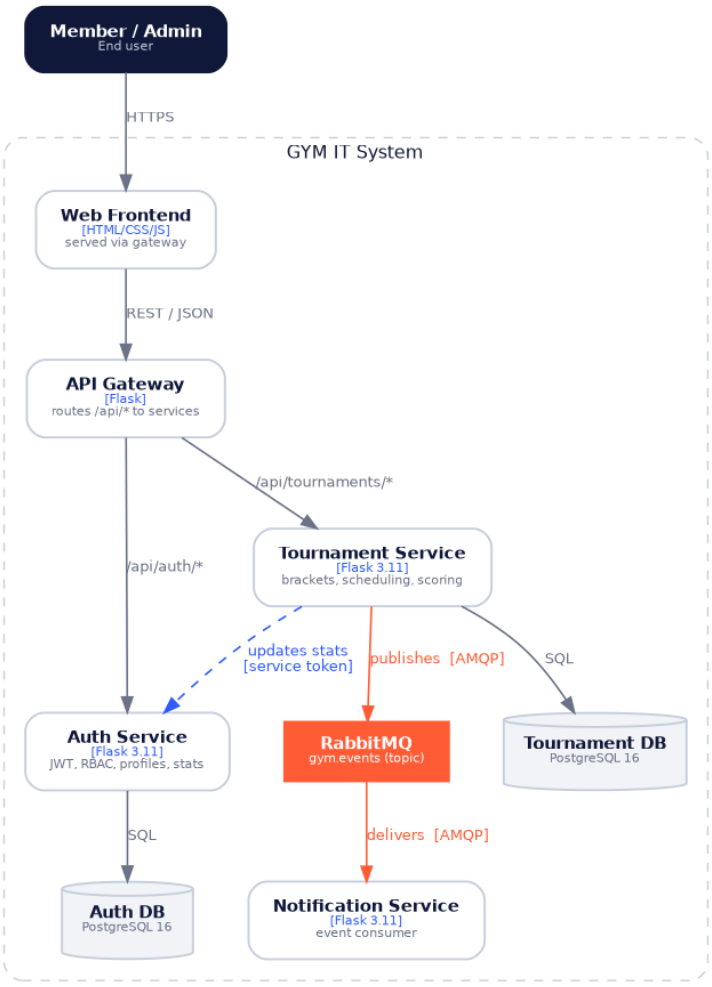

Figure 4 — C4 level 2: container diagram.

Service / component	Tech	Port	Responsibility
API Gateway	Flask	8000	Single entry point; routes /api/* and serves the frontend; CORS.
Auth Service	Flask	8001	Registration, login, JWT, RBAC, profiles, player stats.
Tournament Service	Flask	8003	Tournaments, participants, brackets, scheduling, scoring.
Notification Service	Flask	8004	Consumes RabbitMQ events for notifications / reporting.
RabbitMQ	Topic exchange	5672/15672	Durable asynchronous domain-event broker.
Auth / Tournament DB	PostgreSQL 16	—	One isolated database per data-owning service.

Each backend service is a self-contained Flask application with its own database, mirroring the same domain split used by the monolith's modules. The frontend interacts exclusively with the gateway and is unaware of the underlying service topology, treating the distributed backend as a single interface.

Communication

The services use **hybrid communication**. Synchronous **REST/JSON** is used for request/response operations (frontend → gateway → services, and Tournament → Auth for stats). Asynchronous **events over RabbitMQ** are used to decouple notifications and reporting from the core scoring path: the Tournament service publishes domain events to the durable `gym.events` topic exchange after each successful commit, and the Notification service consumes them from the durable `notification.events` queue.

Figure 5 — Asynchronous event flow through RabbitMQ.

Event envelope and catalog

```
{
  "event_id": "uuid",
  "event_type": "tournament.created",
  "occurred_at": "2026-06-20T10:00:00+00:00",
  "source_service": "tournament-service",
  "correlation_id": null,
  "payload": { }
}
```

Event	Meaning
tournament.created	A tournament was created.
tournament.participant_added	A participant was registered.
tournament.bracket_generated	A single-elimination bracket was generated.
tournament.match_scheduled	A match received a scheduled time.
tournament.match_result_recorded	A final score was recorded.
tournament.completed	The final match ended; a champion exists.

Reliability. Events are published only after the database commit succeeds. If the broker is briefly unavailable, the publisher retries and then logs the failure while the user-facing request still completes, so core scoring stays available and notifications recover independently.

Deployment and DevOps

Docker Compose runs the whole system locally (gateway, services, one PostgreSQL per data-owning service, and RabbitMQ with its management UI). For production, Kubernetes manifests define a dedicated namespace, Deployments and Services per component, the API Gateway with three replicas behind a LoadBalancer, a Horizontal Pod Autoscaler that scales on CPU, and an Ingress that terminates TLS. Every service exposes a `/healthz` endpoint used by Docker and Kubernetes for self-healing restarts, and a GitHub Actions pipeline lints, tests and builds the services on every push.

4.3 Comparison: monolith vs microservices

Both versions enforce the same domain boundaries, but they differ sharply in how those boundaries are realised at runtime:

Aspect	Modular monolith	Microservices
Deployment unit	One Flask application	Independently deployable services
Module / service comms	In-process Python calls	REST (sync) + RabbitMQ events (async)
Data	One database, shared schema	One database per service (isolated)
Scaling	Scale the whole application	Scale each service independently
Fault isolation	A failure can affect the whole app	Failures contained per service
Entry point / routing	Flask blueprints in one process	API Gateway as single entry point
Operational complexity	Low — one app, one DB	Higher — gateway, broker, many DBs
Best fit	Small team, early stage, fast iteration	Independent scaling, fault isolation, team ownership

In short, the monolith is simpler to build, reason about and operate, which made it the right starting point. The microservices version trades that operational simplicity for independent scalability, fault isolation, independent deployment and clearer team ownership — the qualities targeted by this project.

5 Security

Security is implemented consistently across both versions; the microservices version adds gateway-level concerns. The Auth domain is the trust anchor that issues the JWTs every other module or service relies on.

Mechanism	Detail
Password storage	Passwords are never stored in plaintext; they are kept as salted Werkzeug hashes.
Authentication	Stateless JWT (HS256) signed with a secret from the environment; tokens carry subject, role and expiry.
Authorization (RBAC)	A reusable decorator verifies the token and checks the role on every protected endpoint; members receive 403 on admin routes.
Encryption at rest	Phone numbers are encrypted with AES-256-GCM before storage; GCM also provides tamper detection.
Encryption in transit	HTTPS is enforced at the ingress; inter-service traffic stays on the internal network.
Enumeration resistance	Login returns one generic error for both unknown email and wrong password.
Least privilege	Users cannot change their own role; role changes are an admin-only operation.
Secret management	JWT secret, encryption key and service token come from environment / Kubernetes secrets and are never committed.

In the microservices version the API Gateway is the single security boundary for the frontend: it is the one public entry point, while the backend services stay on the internal network. The component view below shows how the Auth service separates these concerns — route handlers protected by the RBAC middleware, which verifies tokens through the JWT engine, a crypto helper for fields at rest, and ORM models for persistence.

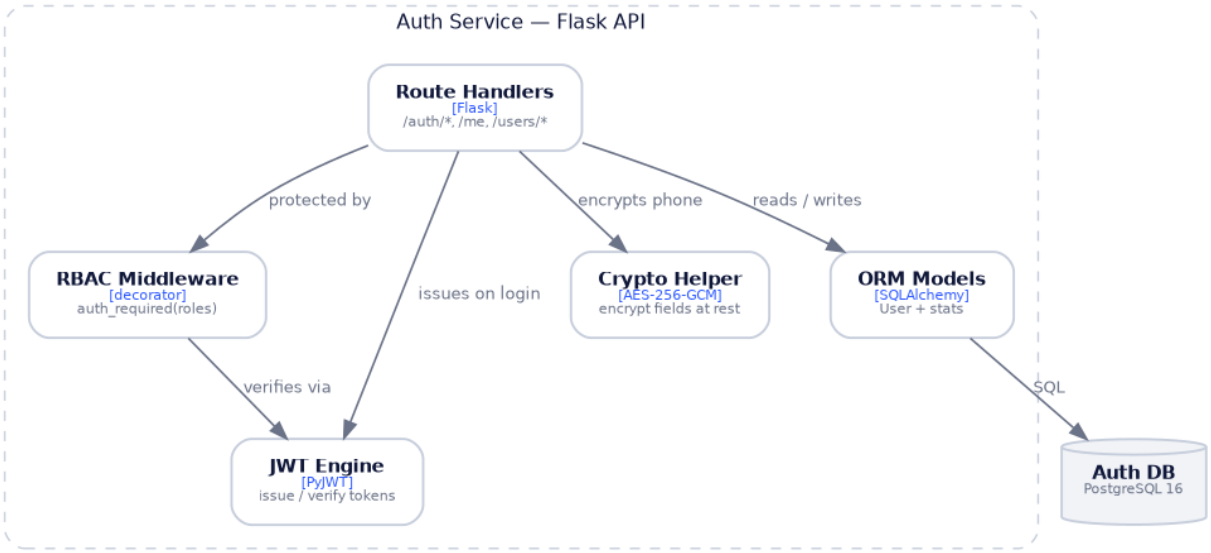

Figure 6 — C4 level 3: Auth service components.

6 Design patterns and code organization

Both versions share the same organisational patterns:

Application factory and blueprints / services

The monolith builds the app with a `create_app` factory that registers one blueprint per module under a stable URL prefix (for example `/api/auth`, `/api/tournaments`). The microservices version applies the same domain split, but each module becomes an independent deployable service behind the gateway.

Layered modules

Each module is layered into a web/route layer (HTTP endpoints), a model layer (SQLAlchemy entities), and domain logic. Bracket generation, for example, is kept as plain functions decoupled from request handling so it can be unit-tested without a server or database.

DTO-style serialization

Database entities are never returned directly. Models expose explicit serialisation (`to_dict`) that controls exactly which fields leave the system — sensitive values such as the password hash are never serialised, and the decrypted phone is returned only on authorised profile and admin flows. This keeps a stable API contract independent of the schema.

Centralized RBAC and consistent errors

Authorisation is a single reusable decorator rather than per-endpoint checks, so the rules are applied uniformly. Both versions return consistent JSON error envelopes (`{"error": ..., "message": ...}`) with appropriate HTTP status codes.

7 Testing and verification

Each version is tested with **pytest**, exercising the domains in isolation against an in-memory or disposable database.

Version / suite	Coverage
Monolith — tests/	Authentication and user management, and the tournament/bracket/scoring flow.

Version / suite	Coverage
Microservices — auth-service	Registration, login, JWT, RBAC (member blocked / admin allowed), AES-256 round-trip — 13 tests.
Microservices — tournament-service	Bracket generation, scheduling, scoring, ties rejected, event envelopes — 19 tests.
Microservices — notification-service	Event-consumer handler behaviour — 2 tests.

In the microservices version a GitHub Actions pipeline runs linting, the per-service test suites and a Docker build check on every push, so regressions are caught before merge. All service tests pass and `docker compose config` validates the local stack.

8 Team and responsibilities

The work is divided so that each member owns one core domain and one architectural quality attribute:

Team member	Core domain	Architectural responsibility
Yeldana Kadenova	Challenges & leaderboards — rankings, points	Simplicity & UI architecture — shared UI, UX, routing
Daniil Glazunov	User management — auth, profiles, roles, staff	Security — JWT, AES-256 encryption, RBAC middleware
Shattyk Kuziyeva	Tournament engine — brackets, scheduling, scoring	Scalability & DevOps — Docker, Kubernetes, CI
Mirgalil Azhmukhambetov	Reporting & notifications — dashboards, events	Fault tolerance & data — reliability, broker, backups

9 Conclusion and future work

Building the GYM IT System twice made the trade-offs between the two architectures concrete. The modular monolith delivered the domain quickly inside a single, easy-to-run application with clear internal module boundaries. Refactoring into microservices preserved those boundaries while adding an API Gateway, a database per service and asynchronous RabbitMQ events, which together provide independent scalability, fault isolation and independent deployment at the cost of additional operational complexity.

Future work includes extending the Notification service with real email / push / WebSocket delivery, extracting a dedicated User/Profile service from the auth domain, adding production observability (metrics, tracing and centralized logging), and strengthening event delivery with an outbox pattern.

Appendix A Microservices API reference

Auth service (via /api/auth)

Method & path	Access	Purpose
POST /auth/register	public	Create an account; returns a JWT.
POST /auth/login	public	Authenticate; returns a JWT.
GET /auth/verify	authenticated	Validate a token; return identity and role.

Method & path	Access	Purpose
GET / PATCH /me	authenticated	View / update own profile.
GET /users	admin	List all users.
PATCH /users/{id}/role	admin	Assign a role.
PATCH /users/{id}/status	admin	Enable / disable an account.

Tournament service (via /api/tournaments and /api/matches)

Method & path	Access	Purpose
POST / GET /tournaments	trainer+ / any	Create / list tournaments.
POST /tournaments/{id}/participants	trainer+	Register a participant.
POST /tournaments/{id}/generate-bracket	trainer+	Generate the bracket.
GET /tournaments/{id}/bracket	any	Read the current bracket.
POST /matches/{id}/schedule	trainer+	Schedule a match.
POST /matches/{id}/score	trainer+	Record a result; winner advances.

Appendix B Repository structure

Monolith (Gymonolith): a single Flask package with one blueprint per module.

```
app/
  auth/ users/ tournaments/ leaderboard/ notifications/ reports/ web/
  models/ config.py security.py extensions.py
migrations/ tests/ docker-compose.yml Dockerfile wsgi.py
```

Microservices (personnna/software-architecture): one folder per service plus shared infra.

```
services/
  api-gateway/ auth-service/ tournament-service/ notification-service/
k8s/ docs/ static/ docker-compose.yml
.github/workflows/ (CI pipelines)
```